

Nanoparticle Building Tutorial

Jahmal Ennis

February 2022

Contents

1	Overview	3
2	Requirements	3
3	Obtaining a structure	3
3.1	Charmm-gui	3
3.2	Other structures	6
4	Mapping a GNP	7
4.1	Making a mapping .ndx	7
4.2	Gromacs mapping tools	7
5	Adding parameters and connectivity	14
5.1	Gold core and Ligand topology	14
5.2	Inserting nanoparticle into a membrane	15
5.3	Running our first nanoparticle simulation	18

1 Overview

This tutorial is designed to walk you through the steps of building your first coarse grained nanoparticle for simulation. There are three necessary steps to build a nanoparticle for simulation: **getting a structure**, **mapping the structure to a coarse-grained structure**, and **adding the topology to your goldcore**

2 Requirements

- 1) Download the Nanoparticle_tutorial_scripts folder from github
- 2) Have an amarel account set up

3 Obtaining a structure

3.1 Charmm-gui

Charmm-gui is a graphical user, hints the gui part of the name, developed to allow users to create biological systems for the purpose of simulation. While there are many other methods to obtain a structure for your nanoparticle charm-gui will be one of the fastest and easiest methods.

0) Go onto charm-gui's webpage and create an account: <https://www.charmm-gui.org/>

1) Once logged in under "input generator", you will click nanomaterial modeller

We are going to be making 2 nm gold nanoparticle with dodecane thiol ligands. The settings should be:

Nanomaterial Type: Gold Nanoparticle with Ligand

$$(AU_XSR_Y)$$

Number of ligands: 1

Ligand structure:



Core Type: Icosahedron

Attachment Type: Staple

Chirality: R

Radius: 10

Coverage: 100

System Type: Vacuum

Select a material from the list below. Currently selected material: **Gold Nanoparticle with Ligand (Au_xSR_y)**

Nanomaterial Type

Ligand Structure

Number of ligands: 1

Ligand structure #1: -S-CH₂- [-CH₂-CH₂- 6] 1 -H Add monomer unit

Core Type

Core Type Icosahedron
Attachment Type Staple
Chirality R
Radius (Å) 10

Ligand Morphology

Coverage (%) 100

System Type

☐ Solvated
☒ Vacuum

References for Nanomaterial Force Fields:

Next Step:
Build Nanomaterial

Click the "Next Step button", this step may take a while depending on the size of the system and number of current users on Charmm-gui.

2) For the next step insert the setting as per tutorial pictures.

Building	Size Determination	PBC Setup	Input Generator
Building Input: step1_nanomaterial.inp Building Output: step1_nanomaterial.out Building PSF: step1_nanomaterial.psf Building PDB: step1_nanomaterial.pdb (view structure) Building Info: step1_nanomaterial.str			
JOB ID: 4546192676 download .tgz			

Box Size Options:

☐ Specify Box Size

☒ Fit Box Size to Material Size

Box type: Rectangular (Currently, the octahedral box is supported only for CHARMM and NAMD)

Enter Edge Distance: 15.0

Next Step:
Determine System Size

Building	Size Determination	PBC Setup	Input Generator
Size Determination Input: step2_solvator.inp Size Determination Output: step2_solvator.out Size Determination PDB: step2_solvator.pdb (view structure) Size Determination CRD: step2_solvator.crd Size Determination Stream: step2_solvator.str			

JOB ID: 4546192676

[download.tgz](#)

System Size:

Box Type	Rectangle		
Crystal Type	ORTHORHOMBIC		
System Size	A	83.496904	Dimension along the A (X) axis
	B	83.4322497	Dimension along the B (Y) axis
	C	83.441418	Dimension along the C (Z) axis
Crystal Angle	Alpha	90	Angle between the axis B and C
	Beta	90	Angle between the axis A and C
	Gamma	90	Angle between the axis A and B

Periodic Boundary Condition Options:

- ☒ Generate grid information for PME FFT automatically
☐ Explicit grid information for PME FFT

X	Y	Z

Next Step: 
Setup Periodic Boundary Condition

Building	Size Determination	PBC Setup	Input Generator
PBC Setup Input: step3_pbcsetup.inp PBC Setup Output: step3_pbcsetup.out PBC Setup Stream: step3_pbcsetup.str PBC Setup PSF: step3_pbcsetup.psf PBC Setup PDB: step3_pbcsetup.pdb (view structure) FFT Calculation: checkfft.py			

Bookmark this [link](#), if you want to comeback to this page

JOB ID: 4546192676

[download.tgz](#)

Force Field Options:

INTERFACE 

Nonbonded LJ Options:

10-12 force switching 

Input Generation Options:

- ☐ NAMD
☒ GROMACS
☐ AMBER
☐ OpenMM
☐ CHARMM/OpenMM
☐ GENESIS
☐ LAMMPS

Equilibration Input Generation Options:

- ☒ NVT Ensemble

Dynamics Input Generation Options:

- ☐ NPT Ensemble
☒ NVT Ensemble
 Temperature: K

Next Step: 
Generate Equilibration and Dynamics inputs

3) Download the structure by clicking the download.tgz button. Open up that file you downloaded and check it out, because you now have a structure!!

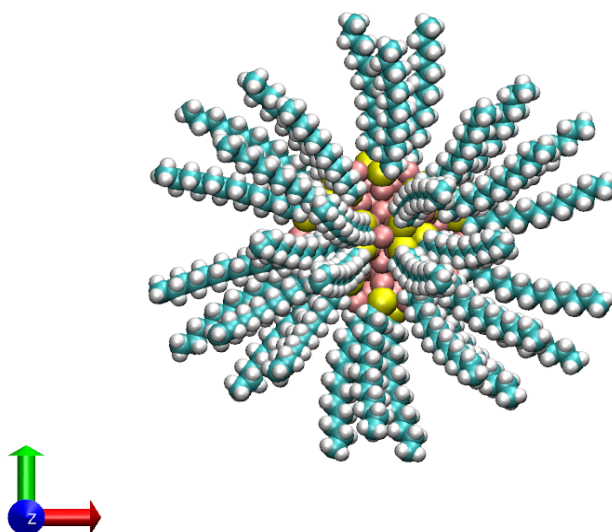
4) **Important!!** You currently have an atomistic version of a gold nanoparticle structure. Before we coarsegrain the structure we should take a look at

it.

Open a terminal and type:

```
cd ~  
cd Downloads  
tar -xzf charmm-gui.tgz  
cd charmm-gui/charmm-gui-(number)/gromacs  
vmd step3_input.gro
```

Your structure should look something like the structure shown below.



Once you have confirmed the structure you can move forward

3.2 Other structures

There are various other methods to obtain a gold nanoparticle beside charmm-gui.

Material Studio:

OpenMD: <https://openmd.org/building-and-equilibrating-a-gold-nanoparticle-in-openmd/>

<https://www.iitg.ac.in/tamalb/karp/gold.html>

VMD FCC lattice

4 Mapping a GNP

4.1 Making a mapping .ndx

In order to map a structure to a coarse-grained one we will first need to write a .ndx file which can be used to map atoms into beads. We will only be mapping the ligands into a coarse-grained version and not the gold core in this tutorial.

Step 1: From the commandline go into the Nanoparticle_Tutorial_Scripts folder and

Run the commands:

```
Python3
import mapping
map1 = mapping.MappingStructure('step3.input.gro','mapping.ndx','dodecanethiol')
map1.controlmap()
```

A file titled mapping.ndx should now be in your file system. Move this file to the folder containing all of your gromacs files.

Step 2: Open the .ndx file using the command:

```
nano mapping.ndx
```

The .ndx file defines which beads should be mapped together. Each line in the .ndx either states the bead number each atom will be associated with or the atom number's which will be mapped together.

Step 3: While this is not a true step, you should note that the .ndx may need to be customized depending on the ligand length or composition. Now exit nano.

4.2 Gromacs mapping tools

The structure you obtain from a charmm-gui is an atomistic structure and can be added to an atomistic membrane and simulated using the charmm-gui output files. However, we are going to be simulating a coarse grained system and therefore we must map this atomistic structure into a coarse grained one using gromacs commands and our .ndx.

Step 1: You need a structure file(.gro) and a structure+mass(.tpr) file to map the beads. While you already have a .gro file you will need to make a .tpr file.

First cd into the folder containing all of your atomistic files (ex. cd charmm-gui/charmm-gui-4546192676/gromacs).

Run the command:

```
gmx grompp -f step4.0_minimization.mdp -o step_3.tpr -c step3.input.gro -r
step3.input.gro -p topol.top -n index.ndx
```

A .tpr file named step_3.tpr should now be in your file system.

Step 2: You will now run the commands to actually map your files to a coarse-grained structure.

Run command:

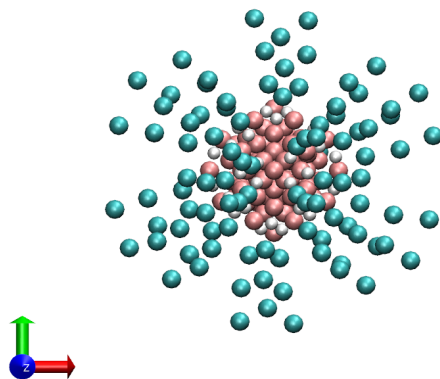
```
seq 0 384 | gmx traj -f step3.input.gro -s step_3.tpr -oxt cg.gro -n map-
ping.ndx -com -ng 385 -e 1
```

Note that the seq command is an iterative command that goes over all of the beads in your.ndx file. In order to know exactly how many beads are in your system I would open the .ndx and go to the final line. Take the line number of the final line and divide it in half then subtract 1. This will account for all the beads in your system and the fact that the iteration starts at 0 and your lines start at 1.

Step 3: You should now have a file called cg.gro. This file contains the structure of your coarse-grained system. Open this structure and check it using :

```
vmd cg.gro
```

You may notice there are white beads where there are suppose to be sulfur beads. This is because these beads are currently being read into VMD as hydrogen beads. Don't worry we are going to fix this.



Step 4: Let's first look into the file to see what the problem is by typing:

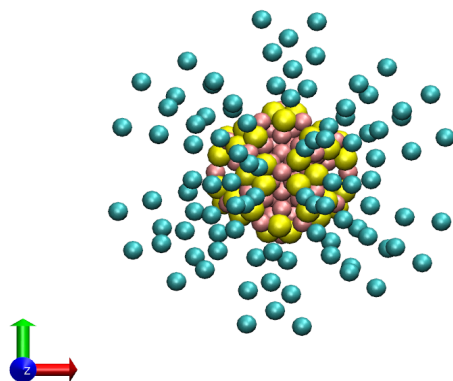
```
nano cg.gro
```

If you scroll through the file you may notice that in the second column, a few different bead types exist (AUC, AUL, AUS, H11, and C1). The problem is that all of the H11 beads are suppose to be S beads instead. We could manually change all of these beads, however this would be time consuming and we would be breaking one of the first lab commandments. Instead let's use some bash commands. Exit nano.

Step 5: The first thing we need to do is switch all of the H11 beads to S beads and we can do this using sed. In the terminal Run the command:

```
sed -i 's/H11/ S/g' cg.gro
```

We have now swapped out all of the Hydrogen beads for sulfur beads. And we can check this using vmd.



Step 7: We are going to now run a command that is going to become useful later when we begin writing our parameters and connectivity for our gold core and ligands. We are going to run:

```
sed -i '/[KY]/{1h;1!H;d};$G' cg.gro
```

This command will rearrange all of the information on ligand beads to the bottom of the page. However, there is one problem with this and it is the fact that it does not move our previous last line to the bottom. The previous last line defines the box shape and size and must be at the bottom of the file. So we are going to break a lab commandment and edit only this line. Open the file using vi and copy the line (it should be somewhere in the middle of the document) and paste it as the final line in your document. Then erase the old line in your document.

58AUL	AUL	344	4.478	4.966	3.681
56AUSL	AUS	353	4.023	4.845	4.008
57AUSL	AUS	354	3.910	4.588	3.662
58AUL	AUL	355	3.871	4.966	3.681
56AUSL	AUS	364	4.175	3.592	3.763
57AUSL	AUS	365	3.758	3.661	3.907
58AUL	AUL	366	3.871	3.377	3.681
56AUSL	AUS	375	4.327	3.498	4.008
57AUSL	AUS	376	4.440	3.755	3.662
58AUL	AUL	377	4.478	3.377	3.681
8.34969	8.34322	8.34414			
59THSLK	S	59	4.563	3.566	4.832
60ETHY	C1	60	4.689	3.219	5.105
62ETHY	C1	61	4.902	2.869	5.470
64ETHY	C1	62	5.177	2.603	5.873
66THSLK	S	63	4.686	4.128	5.002
67ETHY	C1	64	4.923	4.016	5.379
69ETHY	C1	65	5.213	3.981	5.844
71ETHY	C1	66	5.498	4.055	6.316
59THSLK	S	70	4.422	3.971	5.094
60ETHY	C1	71	4.525	3.789	5.503
62ETHY	C1	72	4.665	3.665	6.020

148,2

37%

Make sure to delete any spaces between lines as well.

64ETHY	C1	359	3.877	0.377	3.470
66THSLK	S	360	3.787	4.728	3.471
67ETHY	C1	361	3.661	4.943	3.085
69ETHY	C1	362	3.447	5.250	2.683
71ETHY	C1	363	3.173	5.606	2.357
59THSLK	S	367	3.573	3.506	3.790
60ETHY	C1	368	3.233	3.218	3.677
62ETHY	C1	369	2.885	2.848	3.468
64ETHY	C1	370	2.614	2.455	3.184
66THSLK	S	371	4.139	3.330	3.680
67ETHY	C1	372	4.042	2.921	3.494
69ETHY	C1	373	4.011	2.446	3.220
71ETHY	C1	374	4.076	1.996	2.901
59THSLK	S	378	4.293	3.235	3.927
60ETHY	C1	379	4.257	2.787	3.831
62ETHY	C1	380	4.313	2.258	3.692
64ETHY	C1	381	4.473	1.750	3.536
66THSLK	S	382	4.565	3.565	3.515
67ETHY	C1	383	4.686	3.220	3.237
69ETHY	C1	384	4.898	2.871	2.869
8.34969	8.34322	8.34414			

Step 8: One other problem you may notice in the file is in the third column, all of the bead numbers that tell us the order of the beads in the document are

no longer in order. To fix this we will use a simple trick that works on all types of disorganized files. We are going to use the power of VMD and it's ability to automatically format files. This step is important because gromacs reads in a file that is in a specific format and if the format is off even slightly your simulation will crash. Type:

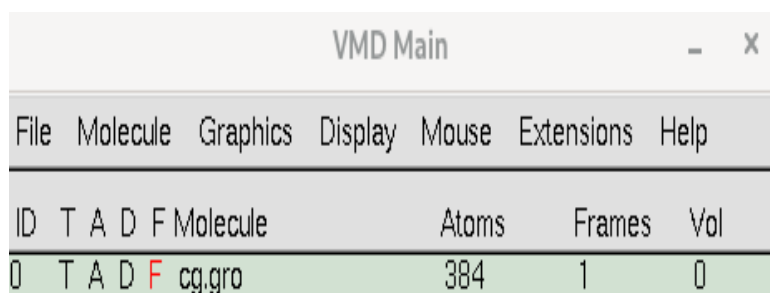
```
vmd cg.gro
```

You should see a structure appear, if you do not see a structure then you probably have not performed step 7 correctly.

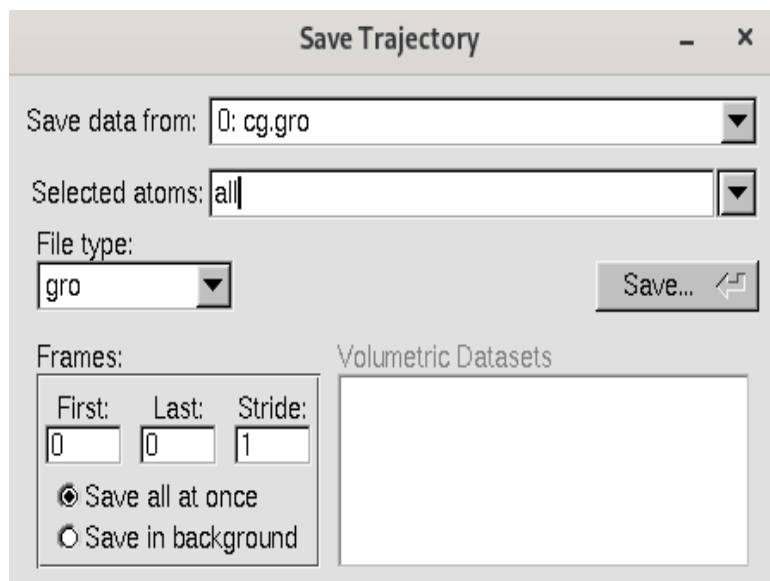
Once you see a structure go to the VMD Main window, click on the first line to make sure it's highlighted green, then click file

VMD Main						
File Molecule Graphics Display Mouse Extensions Help						
ID	T	A	D	F	Molecule	Atoms Frames Vol
0	T	A	D	F	cg.gro	384 1 0

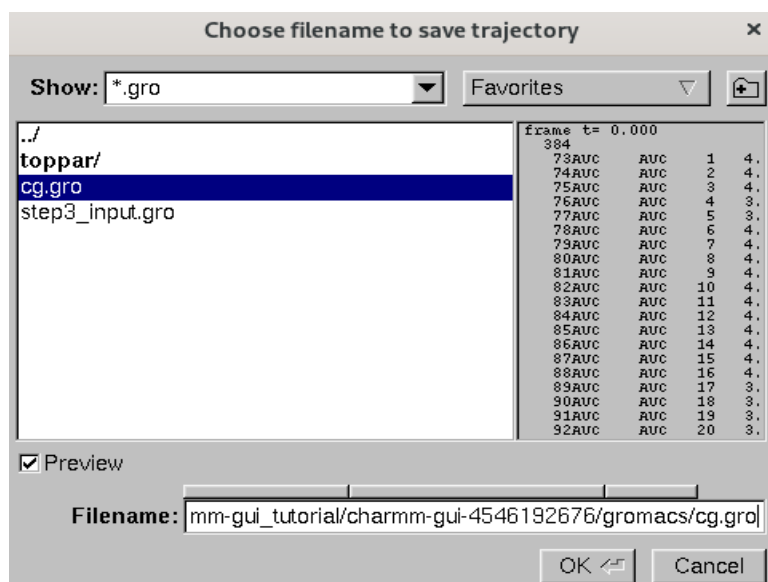




In the drop down menu click "save coordinates" and the save trajectory menu should appear. Set all the setting as the picture below.



Click "save" then click on "cg.gro" and click "OK"



The third column in your file should be fixed now and you can check by using nano.

5 Adding parameters and connectivity

Your coarse-grained structure is complete but we still need to be able to tell gromacs what is connected to what and what kind of interaction each bead is going to have.

5.1 Gold core and Ligand topology

In order to establish a gold nanoparticle that will hold shape we apply an elastic network. We add a series of bonds, many of which may be pseudobonds, in order to hold the gold core completely rigid. The connectivity and bead type of a molecule is known as it's topology and we describe the topology of our system in the .itp files. The script goldbonder3.0.py will write and .itp file for the gold beads in our system.

Step 1: In the terminal write:

```
python 3
import nanoparticlewriter
map1 = nanoparticlewriter.GoldBonder("cg.gro", "cg.itp", ["Qa", "N0", "C1"], [.390, .310, .310],
"dodecanethiol")
```

```
map1.ControlNano()
```

A file called cg.itp should show up in your file system. This file contains all of your topology information for your gold core.

5.2 Inserting nanoparticle into a membrane

We are ready to insert our nanoparticle into a lipid membrane and build our surrounding system. In order to do this we are going to use the insane.py script.

In order to use insane.py you need to have python 2 available on your desktop or on amarel. You will want to move all necessary files to amarel including, all martini files, insane.py, cg.gro, and cg.itp

Step 1 Amarel version: tar -czf Nanoparticle_Tutorial_Scripts.tar.gz Nanoparticle_Tutorial_Scripts

```
scp Nanoparticle_Tutorial_Scripts.tar.gz [netid]@amarel.c.rutgers.edu:/scratch/[netid]
tar -xzf Nanoparticle_Tutorial_Scripts.tar.gz
rm Nanoparticle_Tutorial_Scripts.tar.gz
cd Nanoparticle_Tutorial_Scripts
module load python/2.7.12
```

If you are not using amarel and already have python 2 move on to step 2.

Step 2: Now we will build our membrane. Make sure insane.py is either in the folder you have cg.gro or you write a path to insane.py. Run the command:

```
python insane.py -f cg.gro -pbc hexagonal -d 15 -dz 9 -sol W -solr 0.5 -
o insane.gro -p insane.top -l POPC:1 -center -salt .15 -charge auto -fudge .02
-ring -x 20 -y 20 -z 20
```

A few notes on insane.py:

you can get help with all the commands by typing:

```
python insane.py -h
```

-center = puts nanoparticle in center of membrane
-ring puts lipids in between nanoparticles(in case you have more than 1 nanoparticle in your system)
-fudge= the amount of overlap a nanoparticle and lipid can have (if there is alot of overlap it will cause steric hindrance and your system will crash)

Step 3: look at the system you have created using vmd:

```
vmd insane.gro
```

Try looking at different parts of your system by typing into the graphical representations window:

```
resname AU
resname POPC
name W
```

Make sure each component of your system looks okay before moving on.

Step 5: Now let's go to amarel, move files `cg.itp`, `insane.top`, `insane.gro`, `run_job.main.sh` and `md.i.mdp` to a folder on amarel using commands:

```
[files] scp [netid]@amarel.c.rutgers.edu:/scratch/[netid]/Nanoparticle Tutorial Scripts/filename
```

Our `insane.top` file is our master topology file it can call `.itp`'s and build our entire systems topology. However, we need to modify it to make it run properly. Open the file using:

```
nano insane.top
```

We are going to delete the first `#include` statement in `insane.top` and replace it with these lines:

```
#include "martini_v2.2.itp"
#include "cg.itp"
#include "martini_v2.0_solvents.itp"
#include "martini_v2.0_lipids_all_201506.itp"
#include "martini_v2.0_ions.itp"
```

These statements tell the `.top` file to include these topology files as it constructs our system. We will need to modify these lines somewhat when we begin our simulation in order to add the correct path to these files.

We also need to tell gromacs that there is a nanoparticle in our system instead of a protein. We must change the line under the molecules section from

protein to AU.

```
jje63@ogmios:~/Documents/tutorial — /usr/bin/vim insane.top
```

```
#include "martini.itp"

[ system ]
; name
Protein in INSANE! Membrane UpperLeaflet>POPC=1.0 LowerLeaflet>POPC=1.0

[ molecules ]
; name number
protein          1
POPC             660
POPC             662
NA+              50256
CL-              553
```

```
-- INSERT --
```

9,19

```
jje63@ogmios:~/Documents/tutorial — /usr/bin/vim insane.top
```

```
#include "martini.itp"
```

```
[ system ]
```

```
; name
```

```
Protein in INSANE: Membrane UpperLeaflet>POPC=1.0 LowerLeaflet>POPC=1.0
```

```
[ molecules ]
```

```
; name number
```

AU	1
POPC	660
POPC	662
W	50256
NA+	553
CL-	553

```
-- INSERT --
```

```
9,8 ALL
```

The final topology file should look something like this.

